Converting to External Data to Xacta JSON Using Python

4.0

April 2025

Telos Corporation | 19886 Ashburn Road, Ashburn, Virginia, USA 20147-2358 | 1.800.70.TELOS | 1.800.708.3567 | www.telos.com

© 2025 Telos Corporation. All rights reserved.

Xacta® is a premier suite of cyber risk management and automation software solutions developed by Telos Corporation ("Telos") and launched in August 2000. Xacta content is protected under the copyright laws of the United States of America and is the property of Telos or the party credited as the provider of the content. Copyright © 2000 – 2025. All Rights Reserved.

Xacta has been developed at private expense and embodies certain trade secrets and other confidential, proprietary, and commercially-sensitive information that Telos safeguards through various measures implemented to maintain confidentiality and prevent unauthorized use or disclosure.

Access to and use of Xacta is subject to the terms of a license agreement with Telos (a "License") and is restricted to Authorized Users of the Customer (the "Licensee") that has been granted a License from Telos. **"Authorized Users"** means the employees, consultants, contractors, agents, or other designees of the Licensee who are authorized by the Licensee to use Xacta in accordance with the license granted by Telos and who have been supplied user login keys and credentials by the Licensee or Telos.

If the Licensee of Xacta software solutions and/or documentation is the U.S. Government or an agency thereof, or anyone licensing Xacta on behalf of the U.S Government, the following notice is applicable:

> U.S. GOVERNMENT END USERS: Xacta software solutions and/or documentation delivered to U.S. Government end users are "**commercial computer software**" and "**restricted computer software**" as these terms are used in applicable Federal Acquisition Regulation (FAR) and Defense Federal Acquisition Regulation Supplement (DFARS) clauses, and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation of the Xacta software solutions and/or documentation, and rights in technical data, shall be subject to the terms and restrictions set forth in the License applicable to the Xacta software solutions, and restrictions set forth in applicable FAR and DFARS clauses. See, *e.g.*, FAR 12.211 (Technical Data), FAR 12.212 (Software), FAR 52-227-14 (Rights in Data-General), FAR 52-227-19 (Commercial Computer Software License), DFARS 252.227-7015 (Technical Data – Commercial Items), and DFARS 227.7202-3 (Rights in Commercial Computer Software or Computer Software Documentation). No other rights are granted to the U.S. Government.

No Xacta user or other party may, without the express written permission of Telos or other rights owner, as applicable: 1) reverse engineer or otherwise attempt to discover the source code, object code or underlying structure, ideas, know-how or algorithms of Xacta; 2) copy, reproduce, distribute, publish, display, perform, modify, create derivative works, transmit, or in any way exploit the content of Xacta; 3) distribute any part of Xacta content over any network, including a local area network, or sell or offer it for sale, or; 4) use Xacta source code or content to construct any kind of database or to develop or enhance a competing Governance, Risk and Compliance (GRC) solution. Furthermore, no user or other party may alter or remove any copyright or other notice from Xacta or Xacta content.

Xacta is a registered trademark of Telos Corporation. All other registered trademarks and unregistered common law trademarks that appear in Xacta content are the property of their respective holders.

This document is furnished for informational purposes only. The material presented in this document is believed to be accurate at the time of release. However, Telos assumes no liability in connection with this document except as set forth in the License under which the document is furnished.

Contents

Overview 5
⚠ Prerequisites 5
Step 1: Create Required Folders and Files 5
Step 2: Modify the config.ini File 9
Step 3: Convert the Nmap Files to Xacta® JSON 10
Step 4: Upload the Xacta® JSON data to Xacta.io 12
a) Via Assets List Page 12
b) Via API 14
Appendix A: Configuration Files 17
Appendix B: nmap-xml-to-json.py Script Walkthrough 19
Related Topics 33

© 2025 Telos Corporation. All rights reserved.

Change History

Version Number	Date of Last Revision	Comments
1.0	04/14/2023	New document
2.0	08/29/2023	Removed references to Automated AWS
3.0	5/23/2024	Updated UPI deployment guide name to Deployment Guide Xacta.io – RKE Cluster
4.0	4/16/25	Updated cover page and copyright page

© 2025 Telos Corporation. All rights reserved.

Overview

This article is applicable to the following:

Product	License/ Deployment Type	Release/Build Number	OS	Database
Xacta.io		1.10.2 and higher		

Xacta.io can ingest or import data from external data sources in XML, Excel, CSV, and JSON file formats. Some vulnerability scanners have outputs on XML, JSON, Excel, CSV file formats. These file formats can be converted to Xacta® IO's proprietary JSON data format that is referred to as Xacta® JSON. This article provides information on converting external data in XML format to Xacta.io's proprietary data format Xacta® JSON using Python scripts.

Xacta® JSON format gives users significant flexibility when working with structural datasets, allowing them to transform your data in a variety of ways and offering them limitless possibilities for any asset or vulnerability identification. It will also enable you to upload these data into Xacta.io. For more information on the Xacta JSON data and its associated Xacta.io fields, please refer to the Xacta® JSON Data Dictionary document.

To convert and upload XML files from network scanner such as Network Mapper (Nmap), users must perform the following steps according to the specified order:

- Step 1: Create Required Folders and Files
- Step 2: Modify the config.ini File
- Step 3: Convert the Nmap Files to
- Step 4: Upload the Xacta® JSON data to Xacta.io

⚠ Prerequisites

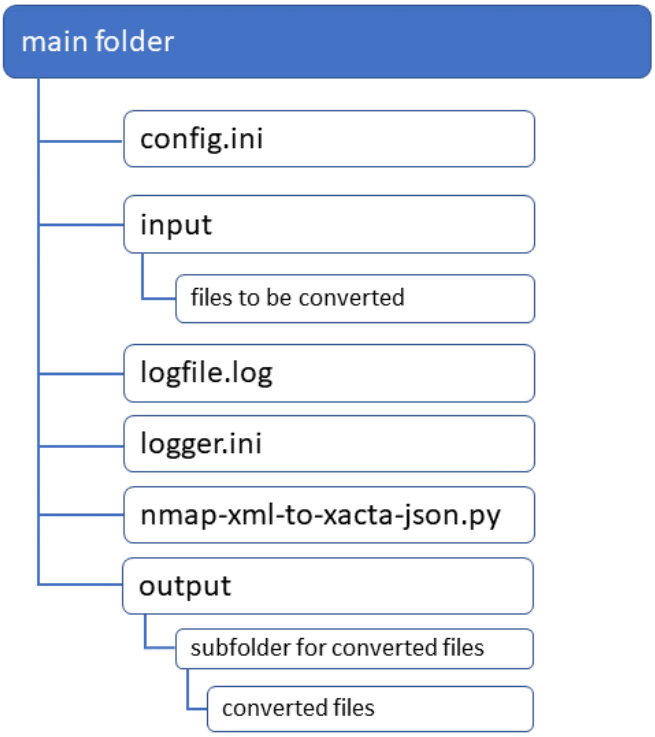
Before converting the XML files, make sure that the following are met:

- Python 3.11 is installed in the machine where the data conversion will be executed. Note that when installing Python 3.11, Python XML plugins and dictionary are included. Please download Python from https://www.python.org/downloads/.
- Copy of the Nmap XML files to be converted to Xacta® JSON. Make sure to save the Nmap XML files in the input folder. See **Step 1: Create Required** for more information on the input folder.
- Xacta.io has been successfully deployed and is up and running. Refer to the Deployment Guide Xacta.io – RKE Cluster for more information.

Step 1: Create Required Folders and Files

© 2025 Telos Corporation. All rights reserved.

The required folders and files should be created to have the following folder structure; this is to ensure that files and folders that are needed by the Python script exist:



Where:

- **main folder:** The main folder will contain or hold all required files and folders for conversion scripts and data to be or have been converted. The main folder can be named according to your organization's preference (e.g., solutions-scripts). Make sure to take note of the folder name as this will be required when performing **Step 3: Convert the Nmap Files to** .
  config.ini: This refers to the configuration file that sets where the Nmap files to be converted are located and where the converted files are saved. It also sets the name of the converted file. See **config.ini** in

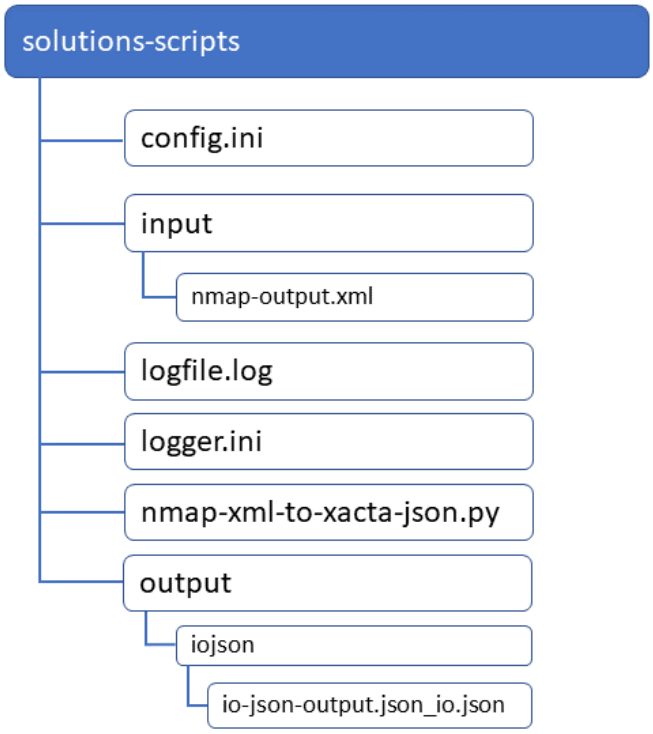
for more information. Note that this file should be saved within the main folder and should be saved in the same location as the logger.ini, input folder, and output folder.

- **input folder:** This refers to the folder where the XML files to be converted are saved. Note that this file should be saved within the main folder and should be saved in the same location as the config.ini, logger.ini, and output folder.
- **logfile.log:** This refers to automatically generated file when the **nmap-xml-to-xacta-json.py** script file is run. This file contains information and error messages generated during runtime.
- **logger.ini:** This contains the logging configuration. This file is used in the nmap-xml-to-xacta-json.py script. See logger.ini in **Appendix A: Configuration Files** for more information. Note that this file should be saved within the main folder and should be saved in the same location as the config.ini, input folder, and output folder.
- **nmap-xml-to-xacta-json.py:** This refers to the python script file used for converting Nmap XML files to Xacta® JSON. See nmap-xml-to-xacta-json.py in the Appendix A: Configuration Files for more information on the content of the script file. Note that this file should be saved within the main folder and should be saved in the same location as the config.ini, logger.ini, input folder, and output folder.

© 2025 Telos Corporation. All rights reserved.

- **output folder:** This refers to the folder where the Nmap XML files converted to Xacta® JSON are saved. Note that this file should be saved within the main folder and should be saved in the same location as the config.ini, logger.ini, and input folder.

  For example, the main folder is named solutions-scripts, the converted file is named as nmap-output.xml, the output subfolder is named as iojson, and the converted file is named as io-json-output.json_io.json:



Perform the steps below to create the required folders and files according to the recommended folder structure provided above:

Note: Steps provided below assume that users are on Linux machine.

a. To create the main folder, launch the Command Line Interface (CLI) terminal and enter the following:

```
$ mkdir [name of the main folder]
```

For example, the main folder name is solution-scripts:

```
$ mkdir solution-scripts
```

Note: *For Windows users, right-click on any location in the machine (e.g., drive C) and select New to create a new folder to create a folder. Then, save the new folder using any preferred name (e.g., solutions-scripts).*

b. To create the nmap-xml-to-xacta-json file, enter the following in the terminal:

```
$ vi nmap-xml-to-xacta-json.py
```


© 2025 Telos Corporation. All rights reserved.

Note: *For Windows users, click File > New from any text editor application. Copy the nmap-xml-to-xacta-json.py content from the* ***Appendix A: Configuration Files*** *and paste the nmap-xml-to-xacta-json.py content on the new text file. Then, save the file in the created main folder (e.g., solutions-scripts folder) with the name of nmap-xml-to-json.py.*

c. Copy the **nmap-xml-to-xacta-json.py** content and paste in the CLI terminal. See **Appendix B: nmap-xml-to-json.py Script Walkthrough** for the nmap script content.
d. Enter the following to save and quit:

```
:wq!
```

e. The nmap-xml-to-xacta-json.py file is now created and listed in the main folder.
f. To create the config.ini file, enter the following in the terminal:

```
$ vi config.ini
```

Note: *For Windows users, click File > New from any text editor application. Copy the config.ini content from the* ***Appendix A: Configuration Files*** *and paste the config.ini content on the new text file. Then, save the file in the created main folder (e.g., solutions-scripts folder) with the name of config.ini.*

For example, the system Id is 23:

```
def asset_import_file_sdk(file_assetScanImport_asset_working,
client_manager=api_manager, system_id=23):
    client_manager.initialize()
    # Get the connector service from client manager
    connector_service = client_manager.get_connector_service()

    # Job request data
    request = AssetScanImportRequest(name='sdk test_asset_import_file Xacta
JSON', system_id=23)

    # The file parameter is the fully qualified path to the asset file
    response = connector_service.import_assets(file=Xacta_Json_file_path,
                                               request_data=request)
```

g. Copy the **config.ini** content and paste in the CLI terminal. See **Appendix A: Configuration Files** for the nmap script content.
h. Enter the following to save and quit:

```
:wq!
```

i. The config.ini file is now created and listed in the main folder.
j. To create the logger.ini file, enter the following in the terminal:

```
$ vi logger.ini
```

Note: *For Windows users, click File > New from any text editor application. Copy the logger.ini content from the* ***Appendix A: Configuration Files*** *and paste the logger.ini content on the new text*

© 2025 Telos Corporation. All rights reserved.

file. Then, save the file in the created main folder (e.g., solutions-scripts folder) with the name of logger.ini.

For example, the system Id is 23:

```
def asset_import_file_sdk(file_assetScanImport_asset_working,
   client_manager=api_manager, system_id=23):
       client_manager.initialize()
       # Get the connector service from client manager
       connector_service = client_manager.get_connector_service()

       # Job request data
       request = AssetScanImportRequest(name='sdk test_asset_import_file
   Xacta JSON', system_id=23)

       # The file parameter is the fully qualified path to the asset file
       response = connector_service.import_assets(file=Xacta_Json_file_path,
                                                  request_data=request)
```

k. Copy the **logger.ini** content and paste in the CLI terminal. See **Appendix A: Configuration Files** for the nmap script content.
l. Enter the following to save and quit:

```
:wq!
```

m. The logger.ini file is now created and listed in the main folder.
n. To create the input folder, enter the following in the CLI terminal:

Note: *For Windows users, go to and right-click in the main folder (e.g., solutions-scripts) and click New to create a new folder. Then, save the new folder as input.*

```
$ mkdir input
```

o. To create the output folder, enter the following in the CLI terminal:

Note: *For Windows users, go to and right-click in the main folder (e.g., solutions-scripts) and click New to create a new folder. Then, save the new folder as output.*

```
$ mkdir output
```

p. After performing steps 1-6, you may proceed to **Step 2: Modify the config.ini File**.

Step 2: Modify the config.ini File

As mentioned, the config.ini file provides information on the locations of the xml files to be converted and the converted files as well as the name of the converted files. To ensure that config.ini points to correct folder locations, the config.ini must be updated accordingly. To modify the config.ini file, do the following steps:

© 2025 Telos Corporation. All rights reserved.

a. Launch the CLI terminal.

Note: *For Windows users, click File > Open from any text editor application and locate the config.ini content. Then, go to step 4 and click Save.*

b. To open the config.ini file, enter the following in the terminal:

```
$ vi config.ini
```

c. Press " i " or press insert key to start editing the config.ini file.
d. Modify the output_folder, and output_filename values:

```
[NMAP-IOJSON]
INPUT_FOLDER=input/[name of the xml file to be converted]nmap-xml
OUTPUT_FOLDER=output/[name of output sub-folder]
OUTPUT_FILENAME=[name of the file converted to Xacta JSON].io-json-output.json
```

For example, the name of the file to be converted is nmap-xml, sub-folder output folder is iojson, and the name of the converted file is io-json-output.

```
[NMAP-IOJSON]
INPUT_FOLDER=input/nmap-xml
OUTPUT_FOLDER=output/iojson
OUTPUT_FILENAME=io-json-output.json
```

e. Press Esc.
f. Enter the following to save the changes:

```
:wq!
```

You can now proceed with converting Nmap XML files to Xacta® JSON format; see **Step 3: Convert the Nmap Files to** .

Step 3: Convert the Nmap Files to Xacta® JSON

Users can simultaneously convert multiple Nmap XML files. Before proceeding with the file conversion, it is important to take note of the following:

- Make sure that **Step 1: Create Required** and **Step 2: Modify the config.ini File** have been completed.
- All Nmap XML files to be converted to Xacta® JSON format have been saved in or copied to the input folder. Users can simply copy and paste the Nmap XML files to the input folder.

© 2025 Telos Corporation. All rights reserved.

To convert to Xacta® JSON, perform the following steps:

a. Launch the terminal for Linux users.
   Note: *For Windows, open the Command Prompt window.*

b. In the terminal, enter the following command to go to the main folder where the nmap-xml-to-xacta-json.py, config.ini, logger.ini, and input and output folders are located:

```
$ cd [directory where the nmap-xml-to-xacta-json script is located]
```

For example, if the main folder name is solutions-scripts:

```
$ cd solutions-scripts
```

c. Enter the following command to convert the Nmap files saved in the input folder:

```
$ python3 nmap-xml-to-xacta-json.py
```

Note: *Depending on your python installation environment, you may also use the following command to convert files to* ***Xacta® JSON*** *format:*

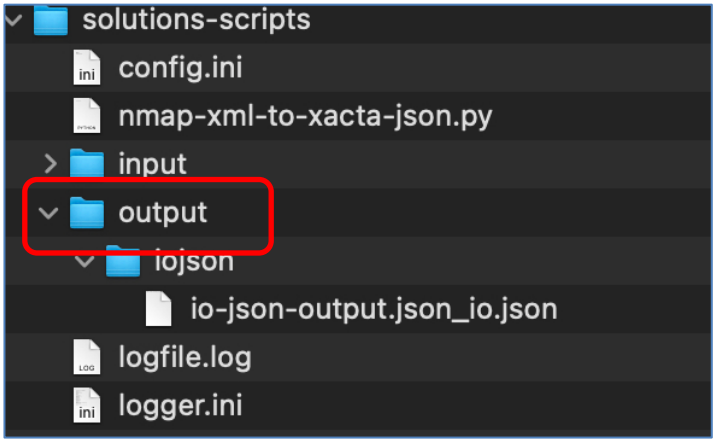
```
$ python nmap-xml-to-xacta-json.py
```

d. Press Enter. A log output information is displayed; for example:

```
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - IP address [redacted].6.34 found on scan <Element 'host'
at 0x1053e36a0>
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - Network Interface {'mac': '', 'ip': [redacted]16.34'}
detected
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - www.example.com
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - report host found: www.example.com
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - Hostname www.example.com detected
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO - Description www.example.com starttime None updated
2023-04-20 14:57:52,463 - Nmap to Xacta.IO Json - INFO -
2023-04-20 14:57:52,464 - Nmap to Xacta.IO Json - INFO -
2023-04-20 14:57:52,464 - Nmap to Xacta.IO Json - INFO -
2023-04-20 14:57:52,464 - Nmap to Xacta.IO Json - INFO - 24 hosts found on 5 nmap scan xml files
2023-04-20 14:57:52,464 - Nmap to Xacta.IO Json - INFO - saving detected devices to io json output
2023-04-20 14:57:52,466 - Nmap to Xacta.IO Json - INFO - Finished running Nmap to IO Json File Generator
```


© 2025 Telos Corporation. All rights reserved.

e. After the script execution is completed, go to the output folder to check and get the converted Xacta® JSON files. The converted Xacta® JSON files can now be imported to Xacta.io; see **Step 4: Upload the Xacta® JSON data to Xacta.io** for more information.



IMPORTANT: *Refer to the Xacta.io JSON Data Dictionary document for the list of data and datasets in an Xacta® JSON file and their associated Xacta.io fields.*

Step 4: Upload the Xacta® JSON data to Xacta.io

There are two ways on how to upload the data to Xacta.io:

a. Via Assets List Page

b. Via API

a) Via Assets List Page

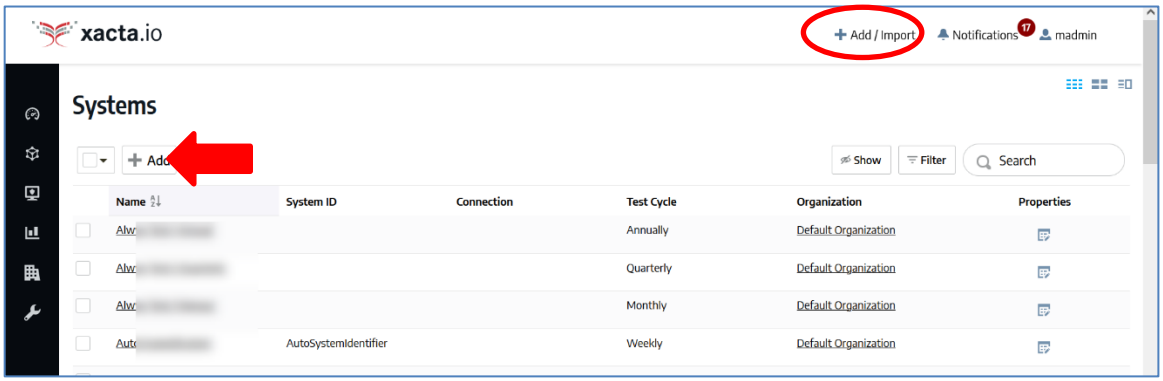
It is recommended to refer to the Importing Assets section of the Xacta.io User Guide for more information. Xacta.io User Guide for more information on importing assets to Xacta.io. The steps provided below assume that users have not created the required organization and system where the asset file in Xacta® JSON format will be imported.

To import assets in Xacta® JSON format, perform the following steps:

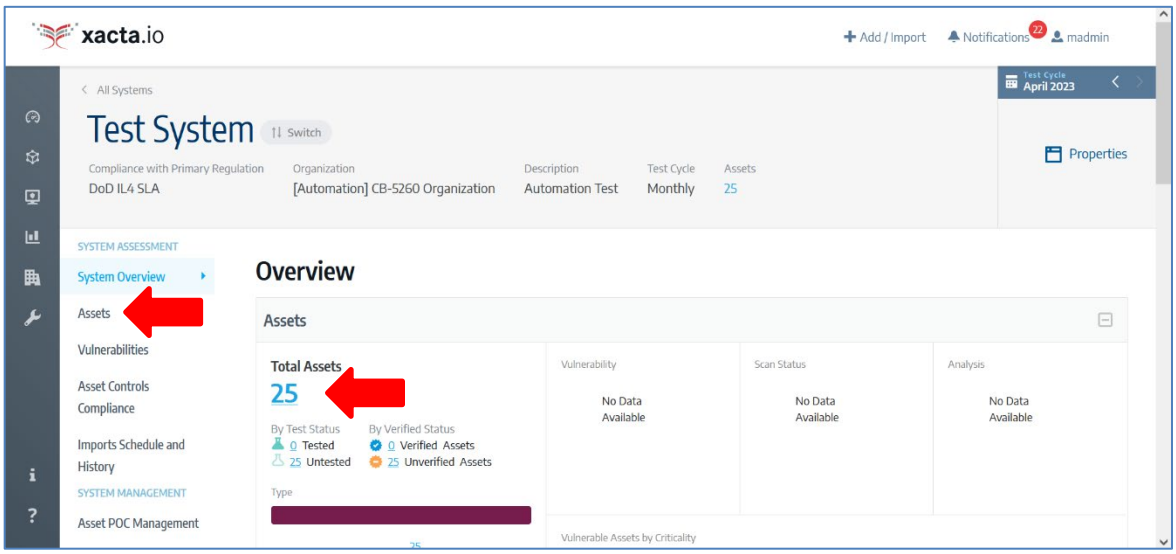
a. Log in to an Xacta.io instance as a Master Administrator.
b. Click Systems > View All. The Systems List page appears.

© 2025 Telos Corporation. All rights reserved.

c. Click Add.
Note: *You may also click Add/Import > New System beside the Notifications icon to create a new system; see encircled item below. See also the Import Policy settings section of the Xacta.io User Guide.*



d. Enter a name for the system. You may enter a maximum of 75 characters. Note that system names must be unique across the application.
e. Enter a unique identifier for the system.
f. From the Organization drop-down menu, select the organization to be associated with the system.
g. From the Regulation drop-down menu, select the regulation(s) to be assigned to the system. This field is optional. If you selected a regulation, proceed to the next step. Otherwise, proceed to step 9.
h. From the Primary Regulation drop-down menu, select a regulation that you want to set as the system's Primary Regulation. Note that if the user only selected one regulation from the Regulation drop-down menu, that regulation will automatically be selected as the Primary Regulation.
i. Enter a description for the system.
j. Select how frequently the system will be tested. The options are Weekly, Monthly, Quarterly, and Annually.
k. Click Save and Close to save your changes and go back to the Systems List page.
l. From the System Overview, click the Total Assets value or click Assets from the System Assessment pane. The Assets List page appears.



m. Click System Import beside the Export button. The Import from drop-down menu.
n. From the Import from drop-down menu, select the Xacta® JSON as the import source.
o. Select the location the assets will be imported from.
p. Click Next. The Cycle pane displays.
q. Select the test cycle to which the assets will be imported from. The options are Current Cycle and Previous

© 2025 Telos Corporation. All rights reserved.

Cycle.

Note: Only user accounts with appropriate permission can import assets into Previous Cycle. If the user does not have permission, the Previous Cycle option will not be displayed.

r. Enter a name for the import job. If this field is blank, the job name displayed in the Import Schedule and History page will display the file name or the network share location.
s. If you want to schedule an import at a specified interval, select the Repeat this import checkbox. Clicking this checkbox allows you to specify the hours, days, weeks, or months the import will be repeated.
t. Click Next. The Summary pane displays.
u. Review the import summary then click Import Now.

b) Via API

Another way of uploading the generated Xacta® JSON file to Xacta.io is to import the file via API; refer to the Xacta.io API Guide for more information.

To import the file via API, perform the following steps:

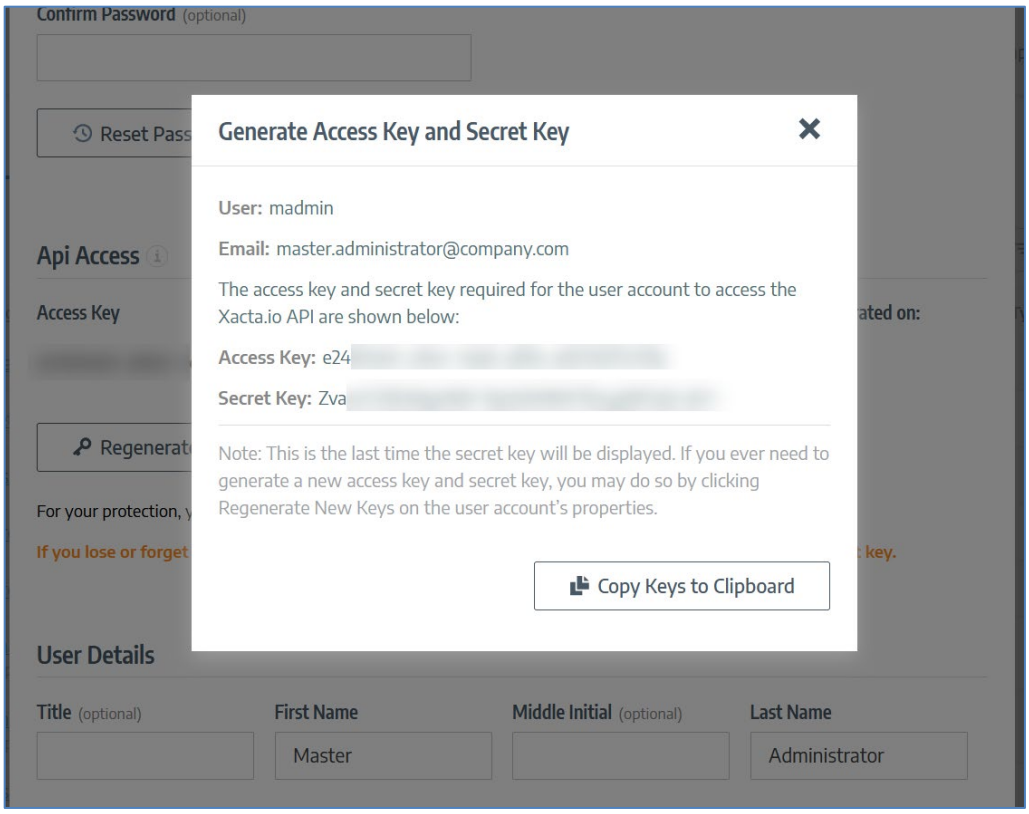
IMPORTANT: *To import assets via API, it is required for the user to generate an access key and secret key to allow the user to have API access. To generate key and access key, please refer to the Xacta.io User Guide. The steps provided below assume that Python 3.11 has been downloaded in the machine where the import will be performed.*

a. Log in to Xacta.io as Master Administrator.
b. Click Administration > User Management > User Accounts. The User Accounts page appears.
c. Click the Login name of the account to be edited. The User Account Properties window appears.
d. Click Edit. Fields of properties that can be edited are displayed.
e. In the API Access section, the generated Access key and the date and time it was created are displayed by default when the API Access setting is enabled. To regenerate keys, go to step 5. To delete keys, go to step 6.

   Note*: If no keys were generated for the user account, the Generate Access and Secret Key checkbox is displayed; select the checkbox to generate an access and secret key for the user's API log in. Then, proceed to the next step.*

f. In the Generate Key and Access Key window, the generated Access and Secret keys are displayed. Make sure to copy the generated keys in a text file and save it in your preferred location because these keys will no longer be retrieved after you close the window. These keys are required when connecting the Xacta.io through API client.

© 2025 Telos Corporation. All rights reserved.

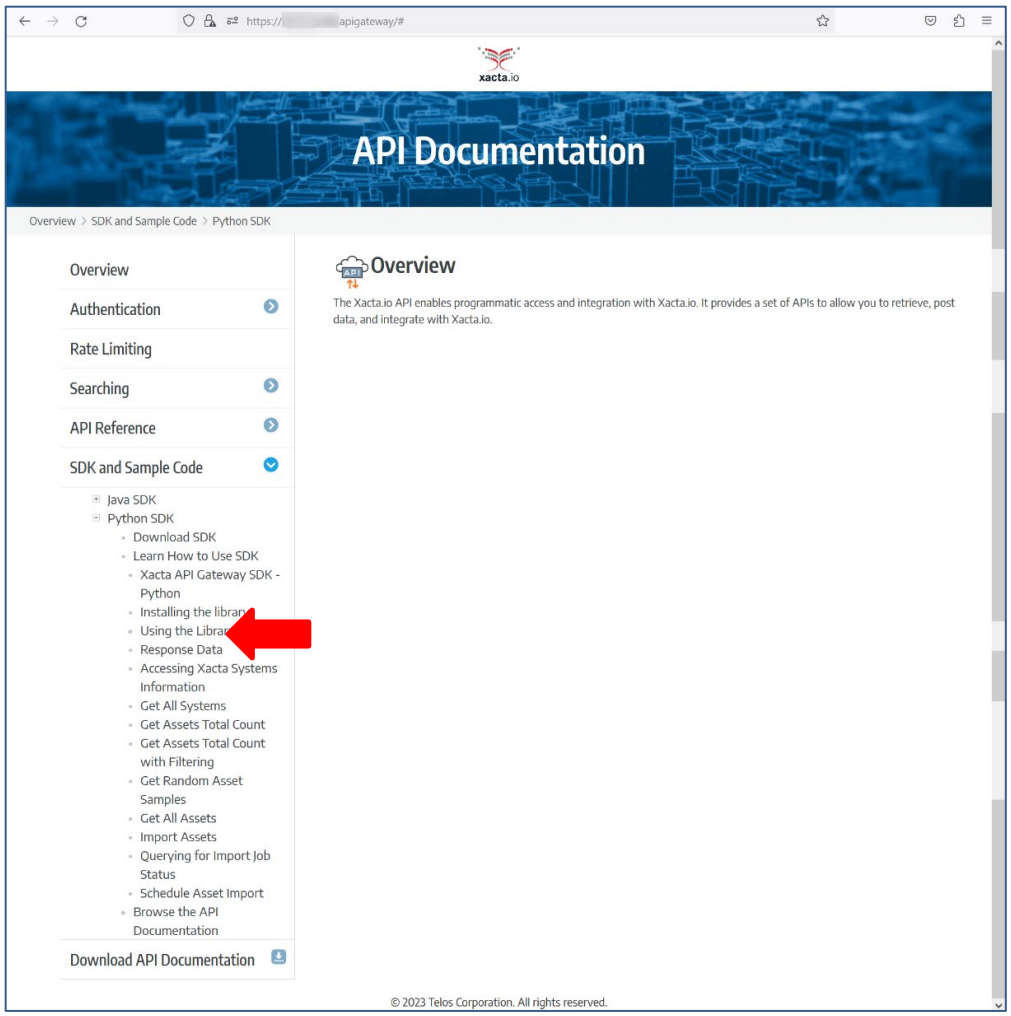


Note: *You may also use the Copy Keys to Clipboard button to copy the Access and Secret Keys as well as the date and time these keys were generated. Open and paste the Access and Secret keys using any text editor application. Then, save the file to your preferred location.*

g. Click the Regenerate New Keys button to generate new Access and Secret keys.
Note*: Regenerating a new access key and secret key will remove the existing access key and the action cannot be undone. Make sure to copy the regenerated keys in a text file and save it in your preferred location because these keys will no longer be retrieved after you close the window.*

h. Click Save Changes. Otherwise, click Cancel.
i. Save the generated Access and Secret keys in a text file and save it in your preferred location because these keys will no longer be retrieved after you close the window. These keys are required when importing Xacta® JSON files in Xacta.io.
j. Open a web browser (e.g., Firefox).
k. In the Address bar, enter the domain name or IP address of Xacta.io instance and append /apigateway. For example, https://xactaiodhs.com/apigateway. The Xacta.io API gateway window appears.
l. From the API navigation pane, locate and click on the **Using the Library** section for more information on importing the APIClientManager, which will be used to import Xacta® JSON files.

© 2025 Telos Corporation. All rights reserved.



m. Run the following part of the python script to upload the Xacta® JSON files converted from XML using the Python SDK:

```
def asset_import_file_sdk(file_assetScanImport_asset_working,
client_manager=api_manager, system_id=[system_id]):
    client_manager.initialize()
    # Get the connector service from client manager
    connector_service = client_manager.get_connector_service()

    # Job request data
    request = AssetScanImportRequest(name='sdk test_asset_import_file Xacta JSON',
system_id=[system_id])

    # The file parameter is the fully qualified path to the asset file
    response = connector_service.import_assets(file=Xacta_Json_file_path,
                                               request_data=request)
```


© 2025 Telos Corporation. All rights reserved.

For example, the system Id is 23:

```
def asset_import_file_sdk(file_assetScanImport_asset_working,
client_manager=api_manager, system_id=23):
    client_manager.initialize()
    # Get the connector service from client manager
    connector_service = client_manager.get_connector_service()

    # Job request data
    request = AssetScanImportRequest(name='sdk test_asset_import_file Xacta JSON',
system_id=23)

    # The file parameter is the fully qualified path to the asset file
    response = connector_service.import_assets(file=Xacta_Json_file_path,
                                               request_data=request)
```

Appendix A: Configuration Files

This section shows lists all the required configuration files and the contents of each configuration file.

config.ini
This file specifies the input and output folder location as well as the output filename for the Nmap Xacta® JSON output file. Below are the contents used for creating the configuration file or config.ini. Copy and paste the contents below to create the config.ini file and follow the steps in **Step 1: Create Required Folders and Files**.

`[NMAP-IOJSON]`
`INPUT_FOLDER=input/[name of the xml file to be converted]nmap-xml`
`OUTPUT_FOLDER=output/[name of output sub-folder]`
`OUTPUT_FILENAME=[name of the file converted to Xacta JSON].io-json-output.json`

logger.ini
Below are the contents used for creating the logger configuration file or logger.ini. Copy and paste the contents below to create the logger.ini file and follow the steps in **Step 1: Create Required Folders and Files**.

```
############# Logger Configuration #############
[loggers]
keys=root,XactaIO_Json

[handlers]
keys=consoleHandler,fileHandler

[formatters]
keys=simpleFormatter,fileFormatter

[logger_root]
level=DEBUG
handlers=consoleHandler
```


© 2025 Telos Corporation. All rights reserved.

```
[logger_XactaIO_Json]
level=DEBUG
handlers=consoleHandler,fileHandler
qualname=XactaIO_Json
propagate=0

[handler_fileHandler]
class=FileHandler
level=DEBUG
formatter=fileFormatter
args=('logfile.log',)

[handler_consoleHandler]
class=StreamHandler
level=DEBUG
formatter=simpleFormatter
args=(sys.stdout,)

[formatter_fileFormatter]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
datefmt=

[formatter_simpleFormatter]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
datefmt=
```


© 2025 Telos Corporation. All rights reserved.

Appendix B: nmap-xml-to-json.py Script Walkthrough

This section details the parts and contents of the nmap-xml-to-json.py script file.

nmap-xml-to-xacta-json.py
This file refers to the python script used for converting Nmap XML files to Xacta® JSON format. Below are the contents of the nmap-xml-to-xacta-json.py file that users will copy and paste into the text file to create the python script for converting XML to Xacta JSON file. Follow the steps in **Step 1: Create Required Folders and Files**. Please refer to succeeding sections for more information on the python script content.

```
import json
import logging
from logging import config
import os
import sys
import xml.etree.cElementTree as xmlElementTree
import configparser

title = "XactaIO_Json"
logging.config.fileConfig('logger.ini')
logger = logging.getLogger(title)

CONFIG = configparser.ConfigParser(interpolation=None)
CONFIG.read('config.ini')

nmap_iojson_input_folder = CONFIG.get("NMAP-IOJSON", "INPUT_FOLDER")
config_iojson_output_folder = CONFIG.get("NMAP-IOJSON", "OUTPUT_FOLDER")
config_iojson_output_filename = CONFIG.get("NMAP-IOJSON", "OUTPUT_FILENAME")


# endregion


def determine_os(elem_osmatches):
    max_accuracy = max(elem_osmatches, key=lambda k: int(k.get("accuracy")))

    if len(max_accuracy) > 0:
        return max_accuracy
    else:
        return None


def get_devices(file):
    logger.info(f"converting file {file}")

    devices = []
```


© 2025 Telos Corporation. All rights reserved.

```
    tree = xmlElementTree.parse(f'{nmap_iojson_input_folder}/{file}'.encode())
    nmaprun = tree.getroot()

    # scanDate
    logger.info(f"file {file} find runstat")
    elem_runstat_finished_time = nmaprun.findall("./runstats/finished")
    scanDate = 0
    sd = elem_runstat_finished_time[0].get("time")
    try:
        scanDate = int(sd)
    except:
        scanDate = 0

    nmapversion = nmaprun.get("version")

    logger.info(f"file {file} output using this Nmap command: {nmaprun.get('args')}")
    hosts = nmaprun.findall("./host")
    for host in hosts:
        device = {}
        net = {}
        os = {}

        device["scanDate"] = scanDate
        device["scannerVersion"] = nmapversion
        elem_address_ipv4 = host.findall("./address[@addrtype='ipv4']")
        # assumes only 1 address element on each host
        address_addr = elem_address_ipv4[0].get("addr")
        if address_addr:
            logger.info(f"IP address {address_addr} found on scan {host}")

        elem_address_mac = host.findall("./address[@addrtype='mac']")
        # assumes only 1 address element on each host
        mac = ""
        try:
            address_mac = elem_address_mac[0].get("addr")
            mac = address_mac
            logger.info(f"Mac address {address_mac} found on scan {host}")
        except:
            mac = ""

        net["mac"] = mac
        net["ip"] = address_addr

        device["network"] = []
        device["network"].append(net)
        logger.info(f"Network Interface {net} detected")

        # hostname
        hostnames = host.findall("./hostnames")
        name = ""
```


© 2025 Telos Corporation. All rights reserved.

```
        try:
            # assume only 1 hostname on each host
            name = hostnames[0].find("./hostname").get("name")
            logger.info(name)
        except:
            name = address_addr

        logger.info(f"report host found: {name}")
        device["hostname"] = name
        logger.info(f"Hostname {name} detected")

        vendor_info = ""
        try:
            vendor_info = hostnames[0].find("./hostname").get("type")
        except:
            vendor_info = name + "unknown_type"

        description = ""
        try:
            description = f"{name} starttime {host.find('starttime')}"
        except:
            description = address_addr

        device["description"] = description
        logger.info(f"Description {description} updated")

        # initailize lists for dictionaries

        device["os"] = {}

        ip = address_addr

        os_name = ""
        os_family = ""
        os_version = ""

        elem_os = host.find("./os")
        if elem_os:
            # assume only 1 os
            elem_osmatch = elem_os.findall("./osmatch")

            if elem_osmatch and len(elem_osmatch) > 0:
                # get the max accuracy value amont each matches
                osmatch_ = determine_os(elem_osmatch)

                if osmatch_:
                    os_name = osmatch_.get("name")

                    elem_osclass = osmatch_.find("./osclass")
                    os_family = elem_osclass.get("osfamily")
                    os_version = elem_osclass.get("osgen")
```


© 2025 Telos Corporation. All rights reserved.

```
                    os_accuracy = elem_osclass.get("accuracy")
                else:
                    os_name = "unknown os"
                    os_family = "unknown os"
                    os_version = "unknown os"
                    os_accuracy = "unknown os"

                logger.info(f"OS {os_name} detected with accuracy {os_accuracy} ")

        logger.info(os_name)
        logger.info(os_family)
        logger.info(os_version)
        if os_family or os_version or os_name:
            if os_name is not "unknown os" or os_version is not "unknown os" or
os_family is not "unknown os":
                os["family"] = os_family
                os["version"] = os_version
                os["name"] = os_name

                device["os"] = os
            else:
                del device["os"]

        devices.append(device)

    return devices


def save_io_json(devices_data, output_folder="output/io-json"):
    logger.info("saving detected devices to io json output")

    if not os.path.exists(output_folder):
        os.makedirs(output_folder)

    hosts = []
    for dev in devices_data:

        if "scannerVersion" in dev:
            scannerVersion = dev["scannerVersion"]
        else:
            scannerVersion = ""

        if "hostname" in dev:
            hostname = dev["hostname"]
        else:
            hostname = "unknown-{IP}"

        if "vendor-info" in dev:
            vendor_info = dev["vendor-info"]
        else:
            vendor_info = ""
```


© 2025 Telos Corporation. All rights reserved.

```
        if "model" in dev:
            model = dev["model"]
        else:
            model = ""

        if "serial" in dev:
            serial = dev["serial"]
        else:
            serial = ""

        if "description" in dev:
            desc = dev["description"]
        else:
            desc = ""

        if "visual-id" in dev:
            vid = dev["visual-id"]
        else:
            vid = ""

        host = {}
        host["dataSource"] = "Nmap"
        host["hostName"] = hostname
        host["scanDate"] = dev["scanDate"]
        host["serial"] = serial
        host["systemModel"] = model
        host["scannerVersion"] = scannerVersion
        netadapters = []
        for a in dev["network"]:
            adapter = {}
            adapter["ip"] = a["ip"]
            adapter["mac"] = a["mac"]
            netadapters.append(adapter)

        host["netAdapters"] = netadapters

        hosts.append(host)

    #     finally we will save the dictionary to an io json file output
    if len(hosts) > 0:
        with open(f"{output_folder}/{config_iojson_output_filename}_io.json", 'w') as
jsonfile:
            try:
                jsonfile.write(json.dumps(hosts))
                jsonfile.flush()
            except Exception as e:
                logger.info("error with json dumps")
                jsonfile.write(host)
    else:
        logger.info(f"No hosts were found on the nmap scan XML input")
```


© 2025 Telos Corporation. All rights reserved.

```
def main(param):
    banner = "Nmap to IO Json File Generator"
    logger.info(f"{banner}")

    logger.info(f"Checking {nmap_iojson_input_folder} folder for nmap files")
    logger.info(f"Preparing to read nmap files")

    all_devices = []

    try:
        cntfiles = 0
        for file in os.listdir(nmap_iojson_input_folder):
            if file not in [".DS_Store"]:
                if file.endswith(".xml"):
                    logger.info(f"Processing {file} xml file detected")
                    devices = get_devices(file)
                    all_devices.extend(devices)
                    cntfiles +=1
                else:
                    logger.info(f"Ignoring folder / file(incorrect file extension)
{file} ")
            else:
                logger.warning(f"Mac directory detected file .DS_Store ignored")
        cntdevices = len(all_devices)
        if cntdevices> 0:
            logger.info(f"{cntdevices} hosts found on {cntfiles} nmap scan xml files")
            save_io_json(all_devices, output_folder=config_iojson_output_folder)
        else:
            logger.info(f"No nmap files found on folder: {nmap_iojson_input_folder}")

    except OSError as e:
        logger.error(f"An error has occured, check if the input directory exists")
        logger.error(f"{e.strerror}")
    finally:
        logger.info(f"Finished running {banner}")

if __name__ == "__main__":
    main(sys.argv[1:])
```

A. *Nmap Hosts Elements*

The hosts element contains all the assets and their important details that we need for an asset identified by the Nmap scan. For example:

```
<nmaprun scanner="nmap"
         args="nmap -A -v -oX sample-03.xml freshmeat.net sourceforge.net nmap.org
kernel.org openbsd.org netbsd.org google.com gmail.com"
         start="1201479002" startstr="Sun Jan 27 21:10:02 2008" version="4.53"
```


© 2025 Telos Corporation. All rights reserved.

```
xmloutputversion="1.01">
    <host starttime="1674699761" endtime="1674700111">
            <status state="up" reason="syn-ack" reason_ttl="0"/>
            <address addr="172.25.5.1" addrtype="ipv4"/>
         <ports>
         </ports>
    </host>
</nmaprun>
```

To obtain all the hosts, we can use the following code:

```
hosts = nmaprun.findall("./host")
```

We can then loop through each host found inside the Nmap run XML root using a for loop:

for host in hosts:

```
#inside the loop we create objects that we will use when building the devices
output function return
```

Within the loop, we can create objects that we will use when building the devices output function return. For example, we can create a device, net, and os object:

```
    device = {}
    net = {}
    os = {}
```

These objects will be used to store the relevant information about each host that we extract from the XML.

Inside the loop we are trying to build an object similar to this one. Most of these "tbd" information are available in an Nmap scan XML

```
dmodel = {
    "hostname": "tbd",
    "vendor-info": "tbd",
    "model": "tbd",
    "serial": "tbd",
    "visual-id": "tbd",
    "description": "tbd",
    "network": [

    ],
    "os": {
        "family": "tbd",
        "name": "tbd",
        "version": "tbd"
    },
    "application": [
        {
            "name": "tbd",
```


© 2025 Telos Corporation. All rights reserved.

```
            "vendor": "tbd",
            "version": "tbd"
        }
    ]
}
```

For example to assign a hostname key value to a device, we use this syntax:

```
device["hostname"] = name
```

Or, to assign a network (key-value object) to the device network list, we use this syntax:

```
device["network"] = [] #we first initialize the object
```

The network object looks like this:

```
{'mac': '', 'ip': '172.31.40.45'}
```

The devices list will contain a list of all device objects found within the XML tree. To build this structure, we will use a Python dictionary. We can use key-value pairs in a Python dictionary to assign values to specific keys. For example:

```
devices = []

# Inside the loop where we extract individual information about each host for
example scandate, hostname and description:

device = {'scanDate': scanDate,
'hostname': hostname,
'description': description,
…}
```

At the end of the loop, here's the resulting device object:

```
#device object
{
  'scanDate': 1674700111,
  'scannerVersion': '7.93',
  'network': [
    {
      'mac': '0A:BC:79:40:E1:37',
      'ip': '172.25.5.9'
    }
  ],
  'hostname': 'scandomain.com',
  'description': ' scandomain.com starttime None',
```


© 2025 Telos Corporation. All rights reserved.

```
  'os': {
    'family': 'Linux',
    'version': '3.X',
    'name': 'Linux 3.1 - 3.2m'
  }
}
```

Finally, we will append each of the devices that we have built inside the loop to the devices list. Then we return the device at the end of the function. The cycle will repeat for every item found on the Nmap XML output file.

```
devices.append(device)
return devices
```

B. Global Variables

The global variables at the top of the nmap-xml-to-xacta-json.py script contains the settings that were configured in the config.ini file. During the initial script run, these variables will be populated with contents from the configuration ini file.

Just like config.ini, the logger.ini is also needed for proper output and logs.

```
title = "Nmap to Xacta® JSON"
logging.config.fileConfig('logger.ini')
logger = logging.getLogger(title)

CONFIG = configparser.ConfigParser(interpolation=None)
CONFIG.read('config.ini')

nmap_iojson_input_folder = CONFIG.get("NMAP-IOJSON", "INPUT_FOLDER")
config_iojson_output_folder = CONFIG.get("NMAP-IOJSON", "OUTPUT_FOLDER")
config_iojson_output_filename = CONFIG.get("NMAP-IOJSON", "OUTPUT_FILENAME")
```

C. Main Function

Like any Python script, the main function serves as the entry point of the program/script. Inside the main function, the program will first loop through each file found in the input directory.

```
try:
    cntfiles = 0
    for file in os.listdir(nmap_iojson_input_folder):

   #will loop through each file found on the input directory
        if file not in [".DS_Store"]:
            if file.endswith(".xml"):
                logger.info(f"Processing {file} xml file detected")
                devices = get_devices(file) # this function will return all
devices found in each file
                all_devices.extend(devices)
                cntfiles +=1
            else:
                logger.info(f"Ignoring folder / file(incorrect file extension)
{file} ")
```


© 2025 Telos Corporation. All rights reserved.

```
            else:
                logger.warning(f"Mac directory detected file .DS_Store ignored")
        cntdevices = len(all_devices)
        if cntdevices> 0:
            logger.info(f"{cntdevices} hosts found on {cntfiles} nmap scan xml files")
            save_io_json(all_devices, output_folder=config_iojson_output_folder)
        else:
            logger.info(f"No nmap files found on folder: {nmap_iojson_input_folder}")

    except OSError as e:
        logger.error(f"An error has occured, check if the input directory exists")
        logger.error(f"{e.strerror}")
    finally:
        logger.info(f"Finished running {banner}")
```

D. *get_devices Function*

This function takes a single parameter, which is the path to a single Nmap XML output. The path should contain a valid XML format with a root nmaprun element. The nmaprun element should have a scanner attribute that contains the name of the scanner used, which is 'nmap.' Additionally, the args attribute shows the Nmap command used for this output XML. These pieces of information can be easily read by Python.

In order to perform any XML operations, we need to import the xmlElementTree from the Python Lightweight XML support. XML is inherently a hierarchical data format, and the most natural way to represent it is with a tree. This module has two classes specifically designed for this purpose:

ElementTree, which represents the entire XML document as a tree

Element, which represents a single node in this tree.

Interactions with the entire document, such as reading and writing to/from files, are usually done at the ElementTree level. Interactions with a single XML element and its sub-elements are done on the Element level.

Element is a versatile container object designed to store hierarchical data structures in memory. It can be described as a cross between a list and a dictionary, where each Element has several associated properties:

'tag', a string containing the element's name.

'attributes', a Python dictionary storing the element's attributes.

'text', a string containing the element's text content.

An Nmap scan output will have the following XML root element.

© 2025 Telos Corporation. All rights reserved.

```
<nmaprun scanner="nmap"
         args="nmap -A -v -oX sample-03.xml freshmeat.net sourceforge.net nmap.org
kernel.org openbsd.org netbsd.org google.com gmail.com"
         start="1201479002" startstr="Sun Jan 27 21:10:02 2008" version="4.53"
xmloutputversion="1.01">

</nmaprun>
```

We can retrieve almost any value inside the XML structure using the .text or the .get method of the Python XML library. For example, if we want to get the version of the Nmap scanner used, we can use the following code:

```
tree = xmlElementTree.parse(f'{nmap_iojson_input_folder}/{file}'.encode())
nmaprun = tree.getroot()
nmapversion = nmaprun.get("version")
```

Another example is retrieving an element inside another element. For instance, consider the following runstats element:

```
<nmaprun …
…
   <runstats>
       <finished time="1201481569" timestr="Sun Jan 27 21:52:49 2008"/>
           <hosts up="8" down="0" total="8"/>
           <!-- Nmap done at Sun Jan 27 21:52:49 2008; 8 IP addresses (8 hosts up)
scanned in 2567.750 seconds -->
</runstats>
</nmaprun>
```

To retrieve the finished element, you can use the find() method of the parent element nmaprun, like this:

```
elem_runstat_finished_time = nmaprun.findall("./runstats/finished")
scanDate = 0
sd = elem_runstat_finished_time[0].get("time")
try:
    scanDate = int(sd)
except:
    scanDate = 0
```


© 2025 Telos Corporation. All rights reserved.

At the start of the script, we parse the entire XML structure and obtain the root using tree.getroot(). This can be achieved by running the following code:

```
tree = xmlElementTree.parse(f'{nmap_iojson_input_folder}/{file}'.encode())
nmaprun = tree.getroot()
```

Once we have the nmaprun variable, we can extract elements from it using the find() or findall() methods. If we want to retrieve multiple elements inside an element, we use findall(). If we want to get a single element inside an element, we use find(). We can also obtain the value of an element using the text attribute. However, since most of the data needed in an Nmap scan is inside an element, we will focus on using find(), findall(), or get("attribute_name").

© 2025 Telos Corporation. All rights reserved.

E. *save_io_json Function*

This function takes several parameters, including devices_data, which is a list of devices gathered from the Nmap XML output, and output_folder, which is the folder where the output file will be saved. When this function is called, it saves all the devices_data into an output file inside the output_folder. The output file will be structured as an Xacta® JSON structure, similar to the following example:

```
[
  {
    "dataSource": "Nmap",
    "hostName": "scanme.nmap.org",
    "scanDate": 1674548304,
    "serial": "",
    "systemModel": "",
    "scannerVersion": "7.93",
    "netAdapters": [
      {
        "ip": "45.33.32.156",
        "mac": ""
      }
    ]
  },
  {
    "dataSource": "Nmap",
    "hostName": "www.example.com",
    "scanDate": 1676902667,
    "serial": "",
    "systemModel": "",
    "scannerVersion": "7.93",
    "netAdapters": [
      {
        "ip": "93.184.216.34",
        "mac": ""
      }
    ]
  }
]
```

In the above example, each device is represented as a dictionary with several keys, such as dataSource, hostName, scanDate, serial, systemModel, scannerVersion, and netAdapters. The netAdapters key contains a list of dictionaries, where each dictionary represents a network adapter with an IP address and MAC address.

Now let us define a blank object called host

```
host = {}
```

Next, we need to assign the key-value pairs in the Xacta® JSON structure. For example, we can assign a dataSource key with a hardcoded value of "Nmap", since all of our hosts came from the Nmap scan output XML. We can do this as follows:

```
host["dataSource"] = "Nmap"
```


© 2025 Telos Corporation. All rights reserved.

Then, we can map all the key-value pairs in the devices_data dictionary that we got from the get_devices function that we discussed earlier. We can map the other fields like this:

```
host["dataSource"] = "Nmap"
host["hostName"] = hostname
host["scanDate"] = dev["scanDate"]
host["serial"] = serial
host["systemModel"] = model
host["scannerVersion"] = scannerVersion
```

To map fields containing a list of objects, such as netAdapters, we can use a loop to iterate over each network adapter in the dev["network"] list and create a new dictionary to represent the adapter with its IP address and MAC address. We can append each adapter dictionary to a new list called netadapters, and then assign this list to the host["netAdapters"] key. Here's an example of how to do this:

```
netadapters = []
for a in dev["network"]:
    adapter = {}
    adapter["ip"] = a["ip"]
    adapter["mac"] = a["mac"]
    netadapters.append(adapter)
host["netAdapters"] = netadapters
```

This way, we can map all the necessary fields in the devices_data dictionary to the corresponding keys in the Xacta® JSON structure.

Finally, we will save the dictionary to a file using file I/O (Input/Output) in Python. We will save the dictionary as an Xacta® JSON file output.

If there are hosts found in the Nmap XML input, we will open a file and write/flush the dictionary into a JSON format output using the json.dumps method. If there is an exception with json.dumps, we will write the host variable to the file. Otherwise, if no hosts were found, we will output a message indicating that no hosts were found in the Nmap scan XML input.

Here's the final code:

```
#     we will save the dictionary to an Xacta® JSON file output
if len(hosts) > 0:
    with open(f"{output_folder}/{config_iojson_output_filename}_io.json", 'w') as
jsonfile:
        try:
            jsonfile.write(json.dumps(hosts))
            jsonfile.flush()
        except Exception as e:
            logger.info("error with json dumps")
            jsonfile.write(host)
else:
    logger.info(f"No hosts were found on the nmap scan XML input")
```


© 2025 Telos Corporation. All rights reserved.

Related Topics

- Xacta.io User Guide
- Deployment Guide Xacta.io – RKE Cluster
- Xacta.io API Guide
- Xacta® JSON Data Dictionary

© 2025 Telos Corporation. All rights reserved.